Electronics & Computing Club

Mini Synth Project (No-Code)

Build & Code: Arduino Workshop

Written by Charles Ross and Alexander D'Alesio on February 24th, 2026

Section 1
Overview

Time for some music! Arduinos are fast devices, capable of changing their pins from 1s to 0s at incredible speeds. We can take advantage of this to generate sounds and play music from it!

To accomplish this, we need to understand what sound is. When dealing with audio in electronics, we can think of sound as a sinusoidal voltage wave, going up and down in a cycle. When we send that wave to a speaker, the speaker will vibrate at the same movement as the wave, making small changes in air pressure that we can hear as sound.

To do this on the Arduino, we can take advantage of the board's quick-switching pins to go from 1, to 0, to 1 again, essentially creating a voltage wave. Then, we can control the speed of that switching, generating different tones and making a full instrument! Sure, we might be missing some of the bells and whistles of a true synthesizer, but this project is a great start if audio electronics interest you!

Section 2

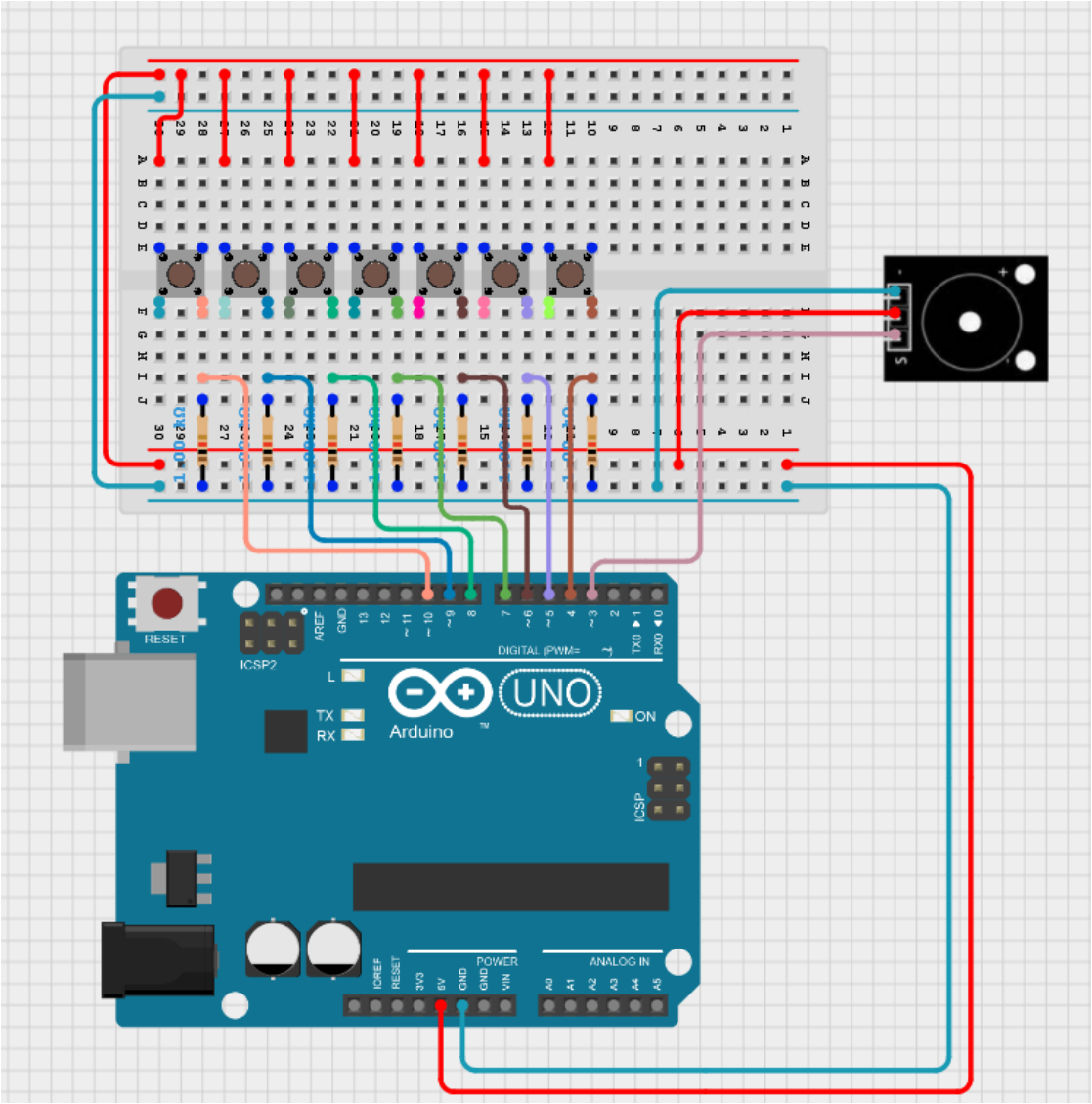
Required Components

Here's a list of what you'll need:

- 7x pushbuttons
- 7x resistors 1kΩ
- 1x passive buzzer
- 1x breadboard
- 1x Arduino Uno
- 1x pack of wires

Section 3

Circuit Assembly



https://app.cirkitdesigner.com/project/24717f9f-0f83-4140-a6fe-b03896dc8d5f

Please refer to the image above for the complete circuit wiring. A link to the online schematic can be found under the image in case you need higher resolution.

If this is your first time using a breadboard (the white rectangle in the image with many holes), please refer to our *Breadboard Guide*.

1. **Start by connecting your 2 power bus lines together**
   The two + lines should be connected with a wire and vice-versa for the two - lines.

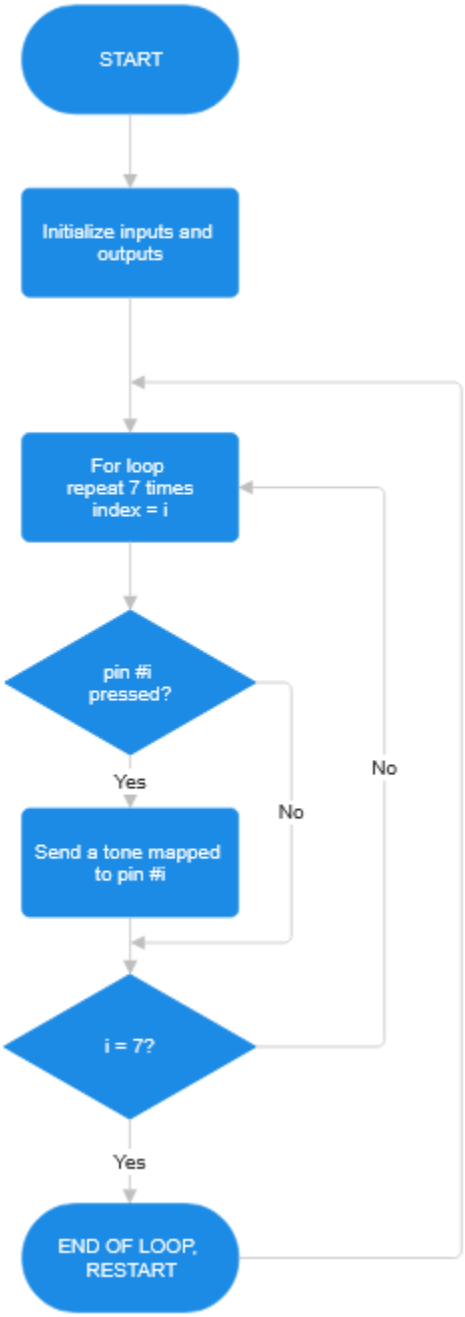
2. **Place your components as indicated by the diagram**
   For all pushbuttons, make sure they are across the middle divot. The resistors should have one leg connected to the - line and the other connected to one of the button pins. The passive buzzer is oriented weirdly on the diagram, so just make sure the three pins are on separate rows (refer to coding dictionary for reference).

3. **Start by wiring the power connections**
   Connect the Arduino 5V pin to the + line and one of the Arduino GND pins to the - line. For the buttons, make sure the pin that you connect to the + line is horizontally opposite to the pin that the resistor is connected to. Connect the middle pin of the passive buzzer to the + line and the - pin to the - line.

4. **Wire up your signal connections to the Arduino**
   Connect Arduino pin 4 to the row where the button and the resistor meet, at the far right button. Repeat this process for pins 5-10, connecting the pin to its respective button. It's important that the pins are properly mapped to the buttons, as this will make coding a lot easier. Connect pin 3 to the S pin of the passive buzzer.

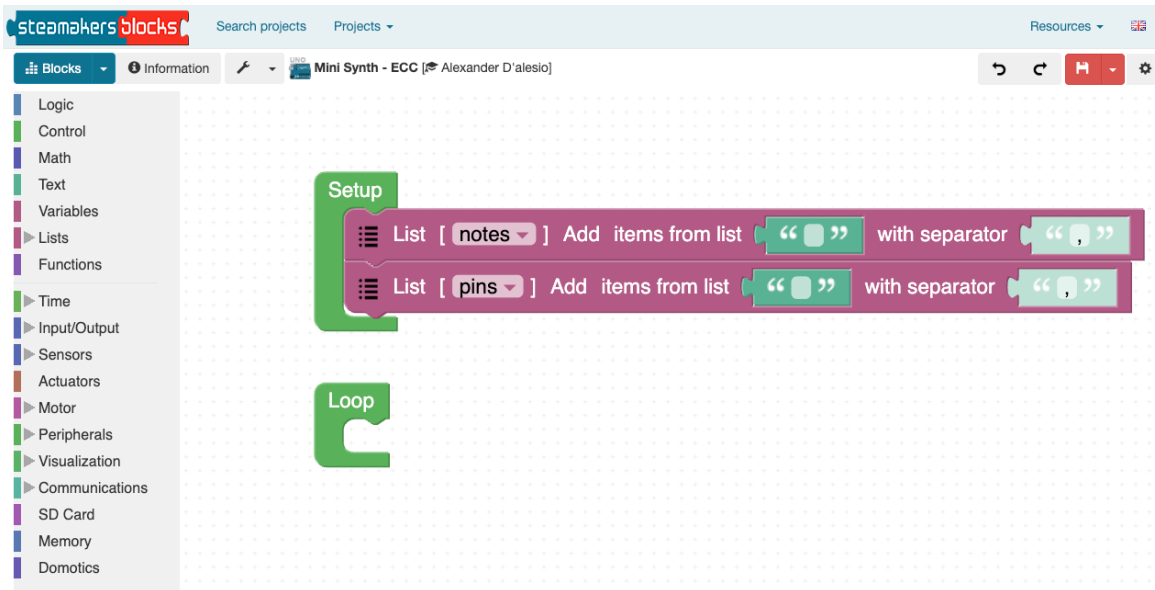
Section 4
Coding

Use this flowchart to guide you while making your program:



Getting Started

First, open the project on STEAMakersBlocks. After registering for a free account, you should see a nearly blank project as shown in the screenshot below.



The site will allow you to choose from many functions and abilities in the categorized sections on the left to add to your workspace. To incorporate them into your program, you can drag and drop them into the green *Setup* and *Loop* sequences. The *Setup* sequence will only run once at the very start of your program, where it will then start the *Loop* sequence and, you guessed it, continue looping forever until you manually stop the program from executing. Your program will execute instructions sequentially, just as the blocks are displayed on-screen (from top to bottom).

Arrays

The two purple blocks contained within the *Setup* sequence are *lists* (STEAMakersBlocks' version of *arrays*). If you aren't already familiar, an array is a type of variable that allows you to store multiple values of the same data type under a single name. Let's examine the example below:

```
const int led[4] = {6, 7, 8, 9};
```

The `const int` portion (which isn't array-specific) is the datatype for this array. The array itself is defined as `led[4]`, where `led` is the array's name and `[4]` is it's size (basically how

many values it can store). The actual data stored is in the form of a comma separated string, enclosed by curly brackets: `{6, 7, 8, 9}`.

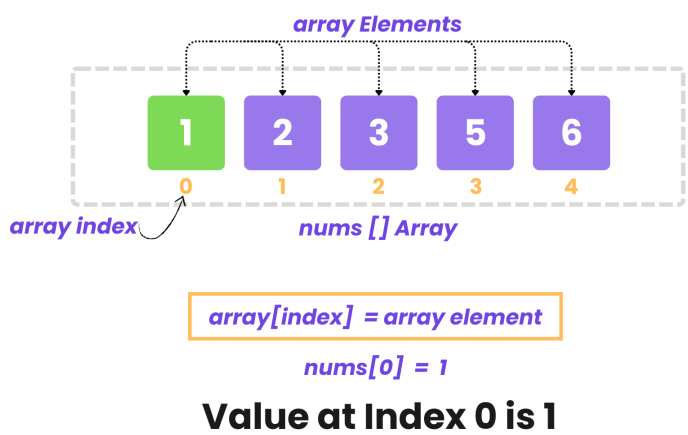
A Practical Example (Arrays)

Let's say we wanted to set a standard integer variable to the second value stored in the array from the previous example. To do this, we can write:

```
int value = led[1];
```

The reason why 1 is written within the square brackets as opposed to a 2 (for the second value) is because array indexing always begins at 0. This is a very important concept and means that the maximum value you would be able to call upon from this specific array is 3.

If you attempt to call upon a value outside the bounds of an array, this leads to serious errors that can lead to memory corruption – so, we generally try to watch out when referencing arrays.



https://read.learnyard.com/content/images/2025/04/index-of--the-given-array.png

Using this newfound knowledge, we can now populate these arrays with actual data. For the *Notes* array, we need to enter the values for 7 frequencies (in hertz, Hz) corresponding to musical notes. While there are 8 notes in an octave, this project uses 7 buttons as "keyboard keys", so we can only play notes from C to B. You can skip over sharp notes (any note marked with a #) and use the values from octaves 4 or 5 (for best results) or any other (up to you – feel free to experiment). Once you obtain the values, enter them separated by commas without any spaces. Refer to the *Note Frequency Chart* on the next page for these frequency values.

NOTE FREQUENCY CHART | HEROIC AUDIO

	Octave 0	Octave 1	Octave 2	Octave 3	Octave 4	Octave 5	Octave 6	Octave 7	Octave 8	Octave 9	Octave 10
C	16.35	32.70	65.41	130.81	261.63	523.25	1046.50	2093.00	4186.01	8372.02	16744.04
C#	17.32	34.65	69.30	138.59	277.18	554.37	1108.73	2217.46	4434.92	8869.84	17739.69
D	18.35	36.71	73.42	146.83	293.66	587.33	1174.66	2349.32	4698.64	9397.27	18794.55
D#	19.45	38.89	77.78	155.56	311.13	622.25	1244.51	2489.02	4978.03	9956.06	19912.13
E	20.60	41.20	82.41	164.81	329.63	659.26	1318.51	2637.02	5274.04	10548.08	
F	21.83	43.65	87.31	174.61	349.23	698.46	1396.91	2793.83	5587.65	11175.30	
F#	23.12	46.25	92.50	185.00	369.99	739.99	1479.98	2959.96	5919.91	11839.82	
G	24.50	49.00	98.00	196.00	392.00	783.99	1567.98	3135.96	6271.93	12543.86	
G#	25.96	51.91	103.83	207.65	415.30	830.61	1661.22	3322.44	6644.88	13289.75	
A	27.50	55.00	110.00	220.00	440.00	880.00	1760.00	3520.00	7040.00	14080.00	
A#	29.14	58.27	116.54	233.08	466.16	932.33	1864.66	3729.31	7458.62	14917.24	
B	30.87	61.74	123.47	246.94	493.88	987.77	1975.53	3951.07	7902.13	15804.26	

https://producelikeapro.com/blog/note-frequency-chart/

For the *Pins* array, we will populate it with the pin numbers on the Arduino that we have buttons connected to. Refer to the circuit schematic for these pins numbers. Once you have found them, enter them as a comma-separated list, leaving no spaces between numbers. Note that you should enter the pin numbers in the order that you would like to play the piano notes – this means that the first pin number entered in the list corresponds to the first frequency entered in the prior list. For a typical keyboard setup, you would want to play the notes C-B from left to right.

Capturing Button Presses

To capture "keyboard key" (button) presses, we will need to constantly check the value at the pins they are connected to on the Arduino. Because of the way these buttons are wired (connected to 5V power), pressing them causes their corresponding pin to go HIGH. This is because, in this instance of digital logic, a HIGH signal (also represented by a binary 1) signifies 5V, while a LOW signal (binary 0) would mean 0V or GND (ground).

A good strategy to accomplish this is to, within the *Loop* sequence, rapidly cycle through all 7 pins and only play a note if a given pin goes HIGH. That means we will need a loop, within our *Loop* (sequence)! Check within the *Control* category on the left-hand side of the screen

for any block that allows you to increment a variable within an adjustable value range. Once you have found it, drag it inside the *Loop* sequence. Given your knowledge on arrays, determine the incrementation range for this block given the fact that we must sweep through 7 pins (*Hint: remember how array indexing works*). Leave the *by* value at its default of 1. This just means that we will be counting upwards, from the first value set until the second value is reached, in steps of 1.

Logic Handling

Remember how we discussed there needing to be a check that verifies the logic level for each pin as the code sweeps through the status of each, one-by-one? Well, now that we've implemented the sweeping, it's time to add the check.

For this, we can use an *if* statement. It is a conditional statement that checks if a certain criteria is met before executing an action. If the criteria is not met, in this case, nothing will happen and the sweeping will continue. This means that we don't require an *else* clause within our conditional statement. With this information, check within the *Logic* section for a block that meets these requirements.

For the conditional statement we want to have checked, we will need to read the value of a pin to evaluate whether it is HIGH (proceed to play a note) or LOW (do nothing/ignore). Search within the *Input/Output* section for a block that can check the value at a pin. It is up to you to determine whether or not these pins are Analog or Digital (one quick search away if you are unsure). There are two versions of the block you are looking for – make sure to grab the larger one with the removable internal block for the pin number.

At this point, our *if* statement won't do us any good, as it is only checking the value of pin 2. In order to properly sweep through the seven pins our buttons are connected to, we need to change the static "2" to a dynamically changing number. Can you think of a way to accomplish this, knowing we have an array of pin numbers, and an incrementing variable named *i*? Once you have a plan, you will find the block you're looking for under the *Lists* > *Numeric* section. After you have added it, replace the static "0" with the proper variable block under the *Variables* section.

Generating the Notes

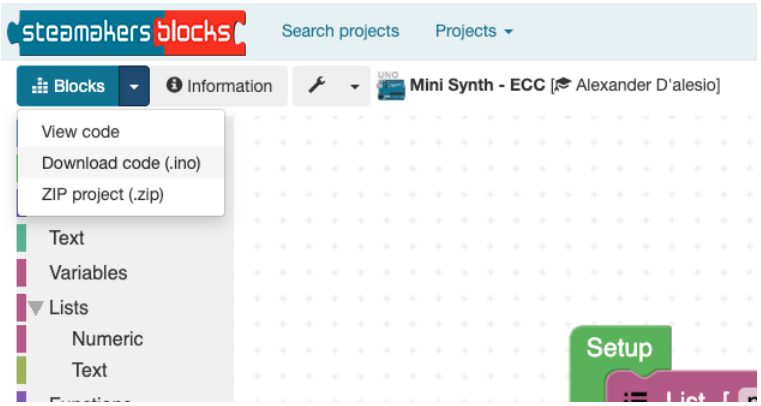
Now for the fun part! To have our buzzer generate musical notes according to the frequencies in the *notes* list/array, we must first add a fundamental block from the *Actuators* section. Once you have found it, set the pin number according to the Arduino pin used by the buzzer as per the circuit schematic and the "Ms" value (note duration in milliseconds) to 250. For the Hz value (frequency to play in hertz), we will need to change the static "1000" to a dynamically changing number. Think about the steps taken earlier with the *pins* array and implement a similar solution for the *notes* array.

The Final Touch

We will add a brief delay so as not to overload the Arduino with constant reading and writing (to the buzzer). This delay will be hardly noticeable as you use the circuit and will be induced at the end of the program each time it runs through the *Loop* sequence. Find the appropriate block under the *Time* category and enter a value corresponding to 0.1 seconds.

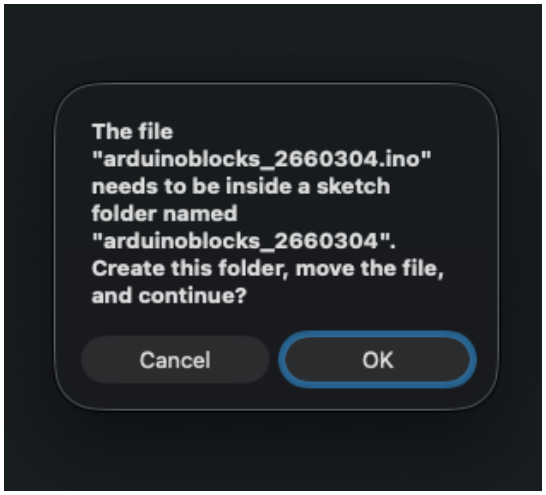
Uploading the Code

To begin the process of uploading your code, click the dropdown arrow next to *Blocks* in the top left of your screen, and then select *Download code (.ino)*.



The resulting file can now directly be opened in the Arduino IDE (you can simply double-click the file as long as you have the software installed). Once, opened, if you are prompted with a warning like the one in the screenshot below, simply click *Ok*. Note that for each time you re-download the .ino file from STEAMakersBlocks, you will need to delete the previous

arduinoblocks folder in your downloads directory (or else the Arduino IDE will complain that a folder with the same name already exists).

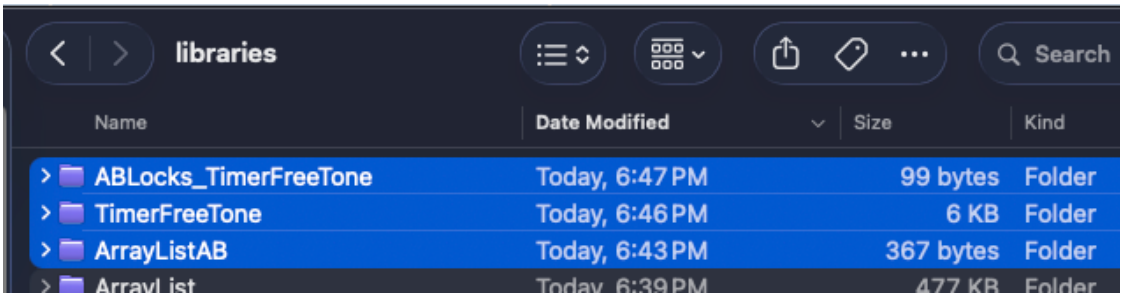


Now, you should see your program (or sketch) written in the C programming language – it has been translated from your previous building blocks.

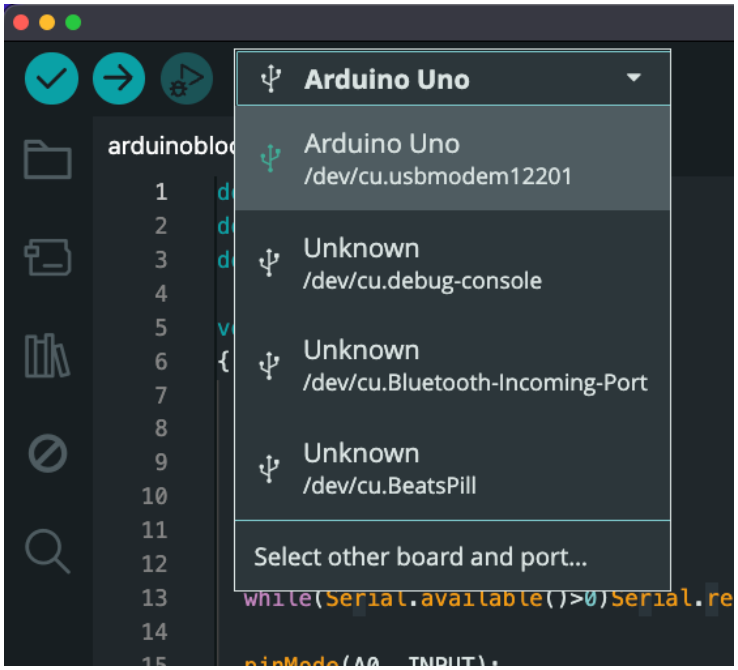
Before uploading the code is to install some pre-requisite libraries. Without these, the code will not compile. Usually, libraries can be installed simply through the Arduino IDE. However, STEAMakersBlocks has used some obscure ones that must be installed manually. To do this, download the *manual-libraries* folder from the project's folder on GitHub. After, you will need to place the three subfolders (do not use the *manual-libraries* folder itself, but its contents) in the correct folder on your computer. Depending on your system, this folder is located at:

- **MacOS**: ~/Documents/Arduino/libraries
- **Windows**: C:\Users*USERNAME*\Documents\Arduino\libraries

Once the folders have been moved or copied to the destination path, it should resemble the screenshot below. Now, restart the Arduino IDE for the libraries to load correctly.



At this point, make sure your Arduino Uno is plugged into your laptop using the provided USB cable. Then, at the top of the screen, ensure *Arduino Uno* is selected as your board.



Now, click the *Upload* button (the → button) at the top left of the screen and wait until your sketch has been completely compiled and uploaded to the board.

Troubleshooting

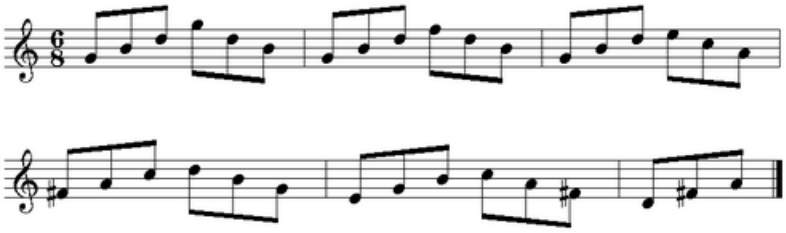
If anything is not working correctly, firstly refer to the circuit schematic to ensure all connections have been properly made. Then, refer to the programming flowchart to make sure your program contains all elements shown in the flow and in that order.

If you are still having issues, don't be afraid to raise your hand or call over a mentor to double-check your work.

Section 5
Challenge Yourself

1. Arpeggiators

One common feature you will find in many synthesizers is the arpeggiator, which automates an arpeggio. An arpeggio is a type of chord where you play each note of the chord one after the other, instead of all at once with a regular chord. This means that you simply play three notes in a sequence going up or down in pitch. This is how they look on sheet music:



So if arpeggios have always existed, what's the point of arpeggiators? Well, what's neat with automating arpeggios is that we can do it really quickly, and make some neat-sounding stuff. I have attached a YouTube short so you can see what arpeggiators look like on modern equipment:

https://youtube.com/shorts/OL4vIdjklrs?si=vY7zGy_tJLUB36-4

If you have noticed, the synthesizer can't play notes at the same time, which is a hardware limitation. That said, we could do the next best thing and have those notes play one at a time, making some neat effects!

Your objective is this: try creating an arpeggiator (going up or down, your choice) by pressing one of the pushbuttons. You can make multiple, and have them all be controlled by the individual buttons.

2. Pitch wheel (but it's a potentiometer)

Another cool effect you can add to your synth is a pitch wheel. The idea of a pitch wheel is to "slide" or "bend" the pitch of your note, gradually changing the frequency according to a literal wheel on the synthesizer that you could turn back and forth.

This is what they usually look like. I also attached a YouTube short you can watch to really hear what they're supposed to sound like.

https://youtube.com/shorts/9rhWNb-P4vg?si=lsorLUTthKWM3O4X

In lieu of a wheel, we can use a simple potentiometer. You would plug the potentiometer's output to an analog pin and use the analog value to define your pitch bend.

The idea is simple: when the potentiometer goes one way, you increase the frequency, and vice-versa. The farther you go one way, the stronger the effect. Try integrating it into your keyboard, and make it so any button you press can get pitch-bent.